STRUCTURAL BREAKS & CHANGEPOINT ANALYSIS

Code and Notebook Review Guide for Project Partners

This is the code-focused companion guide. It explains which Python scripts to inspect or run, how to review the Jupyter notebook cell by cell, what outputs matter, and how to decide whether a run completed correctly.

Important: This guide is based on the project structure and analysis details already established. Filenames below are the expected names from the project handover; first confirm the exact filenames in your local `scripts/` and `notebooks/` folders. Do not run every script blindly.

Project component	Expected location
Empirical notebook	notebooks/real_data_application.ipynb
Analysis scripts	scripts/*.py
Processed CPI data	data/processed/india_cpi_cleaned.csv
Raw CPI workbook	data/raw/cpi_1133.xlsx
Simulation results	results/monte_carlo_raw.csv and results/robustness_raw.csv
Figures	outputs/figures/real_world/ and outputs/figures/simulation/

1. First identify the actual files

Run these from the project root. They only list files; they do not execute analysis.

```
cd ~/STRUCTURAL-BREAKS
find notebooks scripts data results outputs -maxdepth 3 -type f | sort
```

Check that the notebook, Python scripts, input data, results, and figures exist. If a filename differs from this guide, use the filename shown by the command. If a folder is empty or a file is missing, record that rather than substituting a similarly named file.

See the Python scripts and their first lines

```
ls -lh scripts
for f in scripts/*.py; do echo "===== $f ====="; sed -n '1,45p' "$f"; done
```

The first 45 lines often reveal the purpose, imports, input paths, configuration values, and whether the script is a driver or a helper. Also search for the main execution block and output-writing commands:

```
grep -nE '__main__|read_csv|read_excel|to_csv|savefig|OUTPUT|output|N_REP|REPLICATION|n_rep|break' scripts/*.py
```

Do not infer that a script is safe to execute just because it has a .py extension. A script may be a helper imported by another script, or it may launch thousands of replications.

2. Recommended review and run order

Order	File / component	Action	What to verify
1	README.md and requirements.txt	Read; install dependencies in a virtual environment.	Project scope, setup steps, and package imports are consistent.
2	scripts/monte_carlo.py	Inspect configuration; run only if reproducing the simulation.	Five scenarios, replication count, random seed, saved raw results and summary outputs.
3	scripts/robustness_analysis.py	Inspect configuration; run only if reproducing robustness experiments.	14 configurations × 200 replications; factors and output CSV path.
4	scripts/chow_test.py and scripts/bai_perron_dp.py	Inspect whether these are standalone or imported by the notebook/another driver.	Correct break-test setup, selected number and locations of breaks, inference details.
5	scripts/cusum_test.py, quandt_andrews.py, pelt_segmentation.py	Treat as supplementary; run only if their purpose and dependencies are clear.	Supplementary diagnostics are not accidentally presented as primary Monte Carlo results.
6	notebooks/real_data_application.ipynb	Run top-to-bottom in a clean kernel after checking paths and dependencies.	Data preparation, stationarity, growth diagnostics, April 2020 test, DP/BIC result, plots and conclusions.
7	results/ and outputs/	Inspect generated artifacts; compare timestamps and content with current run.	Outputs exist, are non-empty, and correspond to the current settings.

Why the notebook is last in this review sequence

The empirical notebook is the main place to follow the real-data workflow as a coherent analysis. Before running it, confirm that its input data files exist and that any scripts it imports are available. A notebook can also contain copied code rather than calling the scripts directly; inspect its import and file-loading cells to establish the actual dependency.

3. How to review a .py file

Open each script in VS Code or another code editor. Review these areas in order:

Code area	What to look for	Questions to ask
Imports	External libraries and imports from other project scripts.	Are all packages in requirements.txt? Are local imports resolvable from the project root?
Input paths	CSV, Excel, or other files opened by the script.	Do these files exist in the repository? Are paths relative rather than tied to one laptop?
Configuration	Sample size, replication count, break locations, minimum segment length, maximum breaks, random seed.	Do these settings match the project description and the intended experiment?
Core method	Model estimation, test statistic, model selection, segmentation logic.	Does the code implement the method claimed in the README/report?
Output paths	to_csv(), savefig(), file writes, printed summaries.	Are outputs written to the expected results/ or outputs/ folders? Could files be overwritten?
Execution block	if __name__ == '__main__': and function calls below it.	Does running this file start a long experiment, or only define reusable functions?

Script-specific checks

monte_carlo.py — Check that the five scenarios are represented correctly: no break; intercept break; slope break; two coefficient breaks; and variance-only break. The documented design uses 1,000 replications per scenario (5,000 total), sample size 300, and minimum segment length 30. Verify how the code defines a correct detection. In the variance-only scenario, a no-coefficient-break classification is not variance-break detection.

robustness_analysis.py — Check that the design contains 14 configurations × 200 replications (2,800 total). The documented factors are sample size (100/300/500), error SD (0.5/1/2/3), intercept-break magnitude (0.5/1/2/4), and break location (25%/50%/75%). This robustness experiment concerns a single intercept break; do not describe it as testing all break types.

chow_test.py — Check the break date, restricted/unrestricted model definitions, coefficient restrictions, covariance estimator, test statistic, degrees of freedom, and p-value. For the CPI application, the April 2020 result is a HAC-robust joint Wald test, not the textbook classical Chow F-test.

bai_perron_dp.py — Check the segment cost function, minimum segment length, candidate maximum breaks, dynamic-programming recurrence, BIC calculation, and selected break count/locations. The project description calls this Bai–Perron-style dynamic-programming segmentation with BIC, not a complete formal Bai–Perron framework.

CUSUM / Quandt–Andrews / PELT scripts — Identify whether each is a standalone analysis, a helper, or exploratory. Confirm whether it is included in reported results. Do not mix supplementary results into the primary Monte Carlo comparison unless the project methodology explicitly says so.

4. How to review a .ipynb notebook

A Jupyter notebook is a sequence of cells. Read it from top to bottom, because later cells may depend on variables created earlier. In VS Code, open the .ipynb file and use the notebook view. In Jupyter, open it in the browser.

```
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m ipykernel install --user --name structural-breaks --display-name "Python (structural-breaks)"
jupyter notebook
```

Select the project environment as the notebook kernel. Then use Restart Kernel and Run All (or the equivalent command in your editor). A successful run should complete without a red traceback and should reproduce the expected tables/figures. Do not rely only on old saved cell outputs: they may have been generated by an earlier version of the code.

5. Notebook review checklist: cell by cell

Notebook section / cell type	What partners should inspect	What a satisfactory result looks like
Title and introduction	Does it state the question, dataset, period, and purpose of the analysis?	Clearly identifies monthly All-India, Combined, General CPI, Jan 2013–Dec 2024.
Imports and setup	Check imports, warning settings, plotting defaults, and file paths.	All imports succeed; paths work from the repository root; no machine-specific absolute paths.
Data loading	Check the exact file loaded, sheet/columns, date parsing, frequency, and row count.	The intended CPI series loads; 144 monthly observations are accounted for after cleaning.
Cleaning and transformation	Inspect missing values, sorting, duplicates, numeric conversion, and log transformation.	Dates are in chronological order; values are numeric; no unexplained rows disappear.
Time-series plots	Look at the CPI level/log-level and transformed series plots.	Axes, dates, units, and titles are clear; visible movements are discussed accurately.
ADF and KPSS	Check the tested series, deterministic terms, lag settings, statistic, p-value, and critical values.	Interpret both tests carefully; distinguish failure to reject from proof of a unit root/stationarity.
First difference / CPI growth	Check whether the transformation is $\Delta\log(\text{CPI})$ or another growth measure; verify units.	The formula and interpretation match. If multiplied by 100, label as approximate percentage growth.
ACF/PACF and Ljung–Box	Inspect lag choices, confidence bands, Q-statistics, and p-values.	Autocorrelation is described from the diagnostics, not ignored because a model was fitted.
April 2020 break test	Check split/date index, regressors, joint restrictions, HAC covariance, statistic and p-value.	Reported p-value is around 0.0549; not significant at 5%. Call it HAC-robust Wald, not classical Chow F.
DP/BIC segmentation	Check dependent variable, regressors, min segment length, candidate breaks, SSR and BIC table.	BIC selects zero breaks under the tested model; sensitivity at min lengths 24/30/36 also selects zero.
Residual checks / AR alternatives	Inspect residual ACF, Ljung–Box, and any AR(1) or AR(1,12) alternatives.	Persistent residual autocorrelation is acknowledged where present.
Conclusion and limitations	Compare the narrative against numerical outputs and the stated model scope.	Conclusion is cautious: insufficient evidence of breaks under tested specifications, not proof that no breaks ever occurred.

6. Notebook outputs that should be checked and recorded

For every important result, partners should record the cell heading, the exact numerical output, and whether it matches the documented result. If the output differs, do not overwrite the old result or edit the conclusion immediately; first check the input file, kernel, settings, date alignment, and whether the notebook was run from the project root.

Output	Expected project reference	Record
Data dimensions	144 monthly observations in the CPI application.	Rows, columns, date start/end, selected CPI series.
ADF/KPSS	Log CPI plausibly I(1); first difference plausibly I(0), based on the reported diagnostics.	Statistic, p-value, lags, deterministic specification, critical values.
April 2020 Wald test	$p \approx 0.0549$; not significant at 5%.	Test statistic, df, p-value, HAC covariance choice.
DP/BIC	Zero selected breaks under tested specifications.	BIC by candidate break count, selected count, estimated locations if any.
Min segment sensitivity	Min lengths 24, 30, 36 each select zero breaks.	Selected break count for each minimum segment length.
Residual diagnostics	Autocorrelation remains an issue in examined specifications.	ACF/PACF, Ljung–Box results, residual plot, model order.

7. How to run the Python scripts safely

First inspect a script's configuration and execution block. If it is a computational experiment, run a small test only if the script supports a configurable replication count or a safe test mode. Do not edit the main replication count simply to make the script finish unless the run is clearly labelled as a test.

Run from the project root

```
source .venv/bin/activate
python scripts/monte_carlo.py
python scripts/robustness_analysis.py
```

The two run commands above are examples for the named driver scripts. Execute them only after confirming that these files exist, their input/output paths are correct, and you intend to regenerate the experiments. These can take time and may overwrite result files.

Capture terminal output

```
python scripts/monte_carlo.py 2>&1 | tee outputs/monte_carlo_run.log
```

This records terminal output in a log while displaying it. Only use this if the script's output directory exists; otherwise create the directory or save the log elsewhere. A log file is not proof of a correct run: inspect the final summary and generated CSVs.

After a run, check files and timestamps

```
ls -lh results/
ls -lh outputs/figures/simulation/
ls -lh outputs/figures/real_world/
head -5 results/monte_carlo_raw.csv
head -5 results/robustness_raw.csv
```

Check that output files are non-empty, timestamps reflect the run, column names make sense, and row counts match the expected experiment design. For Monte Carlo, the raw file should contain the intended replication-level records; verify the exact row count from the script's output schema rather than assuming a particular number of columns.

8. Common notebook and script problems

Problem	Likely cause	What to inspect
FileNotFoundError	Notebook/script working directory differs from expected root or file missing.	Run pwd; inspect relative paths and confirm the file exists under data/.
ModuleNotFoundError	Wrong Python interpreter/kernel or missing package.	Check selected kernel; run which python and python -m pip show package.
Notebook works only when cells run out of order	Hidden state from an earlier interactive session.	Restart kernel and Run All; move required definitions/imports above their use.
Old output shown in notebook	Saved output from an earlier run.	Restart kernel, clear outputs if needed, then Run All and save updated notebook.
Different numerical results	Different random seed, package versions, data version, sample filtering, or configuration.	Compare inputs, seeds, package versions, parameters, and date/row counts.
Script takes a long time	Large replication count or expensive model/segmentation loop.	Check replications, loops, max breaks, and sample size before interrupting or rerunning.
Figures not saved	Wrong output path or savefig call not reached.	Inspect savefig lines, output directory, and execution trace.

9. Partner review record

For a shared review, each partner can complete this small record and send it to the project lead:

Item	Partner's finding
Notebook opened and exact filename	
Kernel / Python version	
Run All completed? Any errors?	
CPI data row count and date range	
ADF/KPSS outputs checked	

Item	Partner's finding
April 2020 test statistic and p-value	
DP/BIC selected break count	
Scripts inspected / executed	
Result files and figures checked	
Issues / differences from expected results	

10. Final review principles

• Read a script before executing it.
• Run notebooks from a fresh kernel, top to bottom.
• Check the actual data path, sample dates, and observation count.
• Record test statistics and p-values, not only “significant / not significant”.
• Compare generated outputs with the configuration that produced them.
• Keep primary methods separate from supplementary methods.
• Do not claim a variance break was detected from a coefficient-break procedure.
• Do not call the April 2020 HAC-robust Wald test a classical Chow F-test.
• Preserve raw outputs and make a copy before overwriting files.

This guide explains what to inspect and how to interpret the known project workflow. Where the actual source code or notebook differs, the code in the repository is authoritative; note the discrepancy and ask the project lead before changing the analysis.